

— After you finish —

1. Run all smoke / unit tests (`pytest -q` or `pytest tests/smoke`).
 2. git add -A && git commit -m "<brief, present-tense summary>"
 3. Show the tree diff: git diff --stat HEAD~1
-

Task 1: Create a Pipeline for Classifying LLM Errors

Recommended Pipeline Structure:

- **Data Collection:**
 - Curate/generated datasets with annotated examples of different error types.
 - Initial use of synthetic generation by LLMs (GPT-4, Claude Sonnet, Llama 3) to bootstrap labeled datasets.
- **Semantic Embedding:**
 - Generate embeddings (via LLMs, Sentence Transformers, or OpenAI embeddings).
 - Optionally integrate logits from Llama models later.
- **Classification Model:**
 - Logistic Regression or Decision Trees initially (interpretable, baseline models).
 - Multi-class or multi-label classification to handle overlapping error types.
- **Post-Processing:**
 - Model predictions routed to specific re-prompting strategies.

Reference Paper:

- **"Understanding the Dark Side of LLMs Intrinsic Self-Correction"** (Sharma et al., 2023). This paper provides a foundational understanding of error typology in self-correcting LLMs and motivates the semantic approach to classification.
-

Task 2: Setting Up the Classification Workflow

Classification Workflow Outline:

1. Data Preparation & Labeling

- Labels include:
 - Answer Wavering
 - Prompt Bias
 - Overthinking
 - Cognitive Overload
 - Perfectionism Bias

2. Preprocessing Steps

- Text normalization, embedding extraction.
- Tokenization (optional, for token-based approaches).

3. Classification Model Selection

- Start with a baseline Logistic Regression or Decision Tree (for interpretability).
- Consider transformer-based fine-tuning (e.g., DistilBERT, MiniLM) later if necessary.

4. Evaluation

- Precision, Recall, F1-score, accuracy.
- Confusion matrix for analyzing specific misclassification.

5. Feedback Loop

- Incorporate additional annotated examples.
- Experiment with different embedding methods (sentence-transformers, LLM embeddings).

Task 3: Awaiting API Keys for Implementation

- Clarify explicitly in your Claude prompt that API keys from external providers (OpenAI, Claude, Llama via Meta API) will be provided soon, so placeholders or local models (HuggingFace open-source LLM embeddings, such as MiniLM, MPNet) should be used temporarily for prototyping.
- Once keys arrive, easily swap placeholders with real APIs for live inference.

Task 4: Potentially Incorporating Logits from Llama Models

- Prepare workflow to optionally accept logits:
 - When logits become available, include them as additional features alongside semantic embeddings.
 - The workflow should be flexible enough (via simple concatenation or feature fusion) to smoothly integrate logits into your logistic regression or decision tree models.

Missing Information and Solutions:

1. Datasets for Error Classification:

- **Initial Data Generation:**
 - Use GPT-4 or Claude Sonnet to generate synthetic examples based on detailed prompt engineering.
 - Reference: ["Self-Instruct: Aligning Language Model with Self Generated Instructions"](#) by Wang et al. (2022).

Claude Prompt Example (to generate dataset):

You are an expert annotator tasked with creating synthetic examples of Large Language Model (LLM) outputs exhibiting specific errors. Generate concise examples clearly illustrating each error type:

1. **Answer Wavering**: Provide outputs where the model frequently changes its answer or position.
2. **Prompt Bias**: Provide outputs overly influenced by phrasing or suggestive wording of prompts.
3. **Overthinking**: Provide outputs that unnecessarily complicate simple questions or solutions.
4. **Cognitive Overload**: Provide outputs that lose coherence due to overly detailed or excessively broad context.
5. **Perfectionism Bias**: Provide outputs excessively hedging or hesitating, striving for perfect answers unnecessarily.

Generate 5 examples per error type. Format clearly for easy labeling.

2. Specific Error Types from Research Papers:

- Cite explicitly Sharma et al. (2023):
 - **Answer Wavering**: Fluctuation in final answers across self-generated rationales.
 - **Prompt Bias**: Dependence on superficial prompt characteristics rather than intrinsic knowledge.
 - **Overthinking**: Excessive reasoning without additional value.
 - **Cognitive Overload**: Loss of clarity due to overly extensive input/context.
 - **Perfectionism Bias**: Excessive qualifications or hedging to avoid mistakes.
-

3. Appropriate Models for Classification:

- Initial Models: Logistic Regression (scikit-learn), Decision Tree (scikit-learn).
- Advanced/optional: Transformer-based classifiers (DistilBERT fine-tuned, MiniLM).

- Semantic Embeddings: SentenceTransformers or LLM-based embeddings.

Claude Prompt Example (model selection):

Given the objective to classify specific LLM errors based on semantic embeddings of model outputs, compare and briefly justify choosing Logistic Regression vs. Decision Tree classifiers. Focus on interpretability, simplicity, and ease of integration for initial prototyping.

4. Potential Classifiers (like Decision Tree or Logistic Regression):

- **Logistic Regression:**
 - Pros: Simple, interpretable, good baseline.
 - Cons: Limited modeling capacity if feature space becomes complex.
 - **Decision Trees:**
 - Pros: Highly interpretable, intuitive rules.
 - Cons: May overfit; less effective in high-dimensional embedding spaces without careful tuning.
 - Advanced options for later consideration:
 - **Transformer fine-tuning** (DistilBERT, MiniLM).
-

5. Initial Results or Expected Outcomes:

- **Initial Metrics Goals:**
 - Accuracy & F1-score baseline ~70–80% (realistically achievable with clear synthetic examples).
 - Interpretability priority over state-of-the-art accuracy initially.

Claude Prompt Example (expected outcomes):

Outline realistic initial accuracy and F1-score expectations for classifying LLM errors (e.g., Answer Wavering, Prompt Bias) using simple semantic embedding + logistic regression methods. Assume a clearly labeled synthetic dataset. Provide benchmarks or citations if available.

Final Claude Prompt (Summarizing Tasks):

Here's the summarized instruction you can directly use with Claude Sonnet 4 to generate everything required clearly:

You are assisting in setting up a research pipeline for classifying specific errors in Large Language Model (LLM) outputs, inspired by Sharma et al. (2023), "Understanding the Dark Side of LLMs Intrinsic Self-Correction." The specific error categories are: Answer Wavering, Prompt Bias, Overthinking, Cognitive Overload, and Perfectionism Bias.

Complete the following clearly and concisely:

- Pipeline**: Outline the workflow, including data collection, semantic embeddings, baseline classifiers, and post-processing strategies.
- Datasets**: Create a small synthetic dataset (5 examples each) illustrating clearly the five error types.
- Classification Models**: Briefly compare logistic regression and decision trees as baseline classifiers, focusing on interpretability and simplicity.
- Expected Outcomes**: Clearly state initial accuracy, precision, recall, and F1-score expectations for a simple baseline model trained on semantic embeddings and synthetic data. Provide benchmarks or citations if available.

Format your response clearly, ensuring it's directly actionable for immediate prototyping. Leave placeholders for incoming API keys and potential future incorporation of logits from Llama models.

— After you finish —

- Run all smoke / unit tests (`pytest -q` or `pytest tests/smoke``).
 - `git add -A && git commit -m "<brief, present-tense summary>"`
 - Show the tree diff: `git diff --stat HEAD~1`
-

Following this structured approach will enable Claude Sonnet 4 to clearly fulfill your research project goals with actionable, well-cited information ready for implementation.